Top 50 JavaScript Interview Questions

1. What is the difference between var, let, and const?

Feature	var	let	const
Scope	Function scoped	Block scoped	Block scoped
Hoisting	Yes (initialized with undefined)	Yes (but in TDZ until declared)	Yes (but in TDZ until declared)
Re-declaration	Allowed	Not allowed	Not allowed
Re-assignment	Allowed	Allowed	Not allowed
Global object property	Yes (window.varName)	No	No

Example:

```
var a = 10; a = 20;          // allowed
let b = 10; b = 20;          // allowed
const c = 10; // c = 20;     // not allowed
```

2. What is hoisting in JavaScript?

Hoisting is JavaScript's default behavior of moving declarations to the top of their scope before code execution.

- Variable declarations with **var** are hoisted and initialized with undefined.
- **let** and **const** declarations are hoisted but not initialized (TDZ applies).

```
console.log(a); // undefined
var a = 10;
console.log(a); // 10

console.log(b); // ReferenceError
let b = 10;
```

3. What is the Temporal Dead Zone (TDZ)?

The TDZ is the time between the creation of a variable and its initialization when the variable cannot be accessed.

- Applies to **let** and **const**.
- Accessing the variable before initialization throws ReferenceError.

```
console.log(x); // ReferenceError
let x = 5;

console.log(y); // ReferenceError
const y = 10;
```

4. What is the difference between == and ===?

Both are comparison operators.

- == (loose equality): compares values after type coercion.
- === (strict equality): compares values and types without coercion.

```
5 == '5'           // true
5 === '5'          // false
0 == false         // true
0 === false        // false
null == undefined  // true
null === undefined // false
```

5. What are primitive and non-primitive data types in JavaScript?

- **Primitive (stored by value)**: string, number, boolean, null, undefined, symbol, bigint
  Example:
  `let a = 10; let b = a; b = 20; console.log(a); // 10 (unchanged)`
- **Non-Primitive / Reference (stored by reference)**: object, array, function, date, regex
  Example:

```
let obj1 = { x: 10 };
let obj2 = obj1;
obj2.x = 20;
console.log(obj1.x); // 20 (changed)
```

Top 50 JavaScript Interview Questions

@Aman_views

6. What is the difference between null and undefined?

- **undefined:** A variable that is declared but not assigned a value.
  It is JavaScript's default value.
- **null:** An intentional assignment of "no value".
  It is an explicit value set by the developer.

```
let a;                  // undefined
let b = null;           // null

console.log(a);         // undefined
console.log(b);         // null

console.log(typeof a);  // "undefined"
console.log(typeof b);  // "object" (historic bug)
```

7. What is type coercion in JavaScript?

Type coercion is the automatic or implicit conversion of values from one data type to another.

It happens in operations involving different types.

```
"5" + 2         // "52" (string concatenation)
"5" - 2         // 3    (string coerced to number)
true + 1        // 2    (true -> 1)
false + 1       // 1    (false -> 0)
[] + 1          // "1" ( [] -> "" )
[1] + 1         // "11" ( [1] -> "1" )
```

8. What is the difference between typeof and instanceof?

Feature	typeof	instanceof
Purpose	Returns the type of a value.	Checks if an object is an instance of a constructor.
Returns	A string.	A boolean (true or false).
Works with	All data types.	Objects and functions (not primitives).
Example	`typeof 42 // "number"` `typeof "hello" // "string"`	`[] instanceof Array // true` `{} instancecf Object // true`

9. What is the difference between pass-by-value and pass-by-reference in JavaScript?

- **Pass-by-value:**
  Primitive values are passed by value. A copy of the value is passed, so changes inside the function do not affect the original value.
- **Pass-by-reference:**
  Objects (and arrays, functions) are passed by reference.
  The reference (memory address) is passed, so changes inside the function affect the original object.

```
// Pass-by-value (primitive)
function change(x) {
   x = 20;
}
let a = 10;
change(a);
console.log(a);  // 10 (unchanged)

// Pass-by-reference (object)
function changeObj(obj) {
   obj.x = 20;
}
let obj = { x: 10 };
changeObj(obj);
console.log(obj.x);  // 20 (changed)
```

10. What is the difference between shallow copy and deep copy?

- **Shallow Copy:**
  Copies top-level properties only. Nested objects/arrays are copied by reference.
- **Deep Copy:**
  Copies everything recursively. Nested objects/arrays are also copied by value.

```
// Shallow copy
const original = { a: 1, b: { c: 2} };
const shallow = { ...original };
shallow.b.c = 10;
console.log(original.b.c);  // 10 (affected)

// Deep copy
const deep = JSON.parse(JSON.stringify(original));
deep.b.c = 10;
console.log(original.b.c);  // 2 (not affected)
```

Top 50 JavaScript Interview Questions

@Aman_views

11. What is scope in JavaScript? Explain lexical scope.

- Scope determines the accessibility (visibility) of variables.
  JavaScript has Global scope, Function scope, and Block scope.
- Lexical scope (static scope) means a variable's scope is determined by its position in the source code.
  Inner functions have access to variables of their outer (enclosing) functions.

```
const x = 'global';
function outer() {
   const y = 'outer';
   function inner() {
      const z = 'inner';
      console.log(x, y, z); // global outer inner
   }
   inner();
}
outer();
// Output: global outer inner
```

12. What is a closure in JavaScript?

- A closure is a function that remembers (closes over) the variables from its lexical scope even after the outer function has finished execution.
- Closures allow data privacy and create powerful patterns in JS.

```
function outer(a) {
  let count = 0;
  return function inner() {
     count += a;
     console.log(count);
  };
}
const fn = outer(5);
fn();  // 5
fn();  // 10
```

13. What are practical use cases of closures?

1. Data privacy / Encapsulation
2. Function factories
3. Memoization
4. Maintaining state
5. Event handlers & callbacks

```
// Example: Memoization using closure
function memoize(fn) {
   const cache = {};
   return function (...args) {
      const key = JSON.stringify(args);
      if (cache[key]) return cache[key];
      const result = fn(...args);
      cache[key] = result;
      return result;
   };
}
```

14. What is the difference between a function declaration and a function expression?

	Function Declaration	Function Expression
Syntax	`function sum(a, b) { return a + b; }`	`const sum = function(a, b) { return a + b; };`
Hoisting	Yes (can be called before declaration)	No (cannot be called before assignment)
Name	Has a name	Can be anonymous or named
Use case	When function needs to be used throughout the scope	When function is used as a value (e.g., callbacks, assignments)
Example	`sayHello(); function sayHello() { console.log('Hi'); }`	`// TypeError sayHi(); const sayHi = function() { console.log('Hi'); };`

15. What are first-class functions in JavaScript?

- In JavaScript, functions are first-class citizens.
- This means functions can be:
  - Assigned to variables
  - Passed as arguments to other functions
  - Returned from other functions
  - Stored in data structures (arrays, objects)

```
// Assign
const greet = function() { return 'Hello'; };

// Pass as argument
function say(fn) { console.log(fn()); }
say(greet);  // Hello

// Return from function
function createMsg() {
  return function() { return 'Hi'; };
}
const msg = createMsg();
console.log(msg());  // Hi

// Store in array
const arr = [function() { return 1; }, () => 2];
console.log(arr[0](), arr[1]());  // 1 2
```

Top 50 JavaScript Interview Questions

@Aman_views

16. What are higher-order functions?

- A higher-order function is a function that:
  - Takes one or more functions as arguments, or
  - Returns a function as its result.
- Examples: map(), filter(), reduce(), forEach(), setTimeout(), etc.

```
// Function that takes another function as argument
function operate(a, b, fn) {
   return fn(a, b);
}
const add = (x, y) => x + y;
console.log(operate(5, 3, add)); // 8

// Function that returns another function
function multiply(x) {
  return function (y) {
    return x * y;
  };
}
const double = multiply(2);
console.log(double(10)); // 20
```

17. What is an immediately invoked function expression (IIFE)?

- An IIFE is a function that runs as soon as it is defined.
- It helps create a private scope and avoids polluting the global scope.
- Syntax:
  (function() { /* code */ })();
  or
  (() => { /* code */ })();

```
(function () {
   const message = 'IIFE running';
   console.log(message);
})(); // IIFE running

((a, b) => {
   console.log(a + b);
})(5, 7); // 12
```

18. What is currying in JavaScript?

- Currying is a technique of transforming a function with multiple arguments into a sequence of functions, each taking a single argument.
- It helps in function composition and reusability.

```
function add(a) {
  return function (b) {
    return function (c) {
      return a + b + c;
    }
  }
}

console.log(add(1)(2)(3)); // 6
```

19. What is the difference between arrow functions and regular functions?

Feature	Regular Function	Arrow Function
Syntax	function() {}	() => {}
this Binding	Dynamic (depends on how it is called)	Lexical (inherits from surrounding scope)
arguments Object	Available	Not available
Use as Constructor	Yes	No
prototype Property	Yes	No
Best for	Methods, constructors, event handlers	Callbacks, short functions

```
// Regular function
const obj = {
  name: 'Aman',
  sayHello: function() {
     console.log(this.name);  // Aman
  }
};
obj.sayHello();

// Arrow function
const obj2 = {
  name: 'Aman',
  sayHello: () => {
     console.log(this.name); // undefined
  }
};
obj2.sayHello();
```

20. How does this work in JavaScript?

- The value of this depends on how a function is called, not where it is defined.
- Rules:

1. Global context: this refers to global object (window in browser)
2. Object method: this refers to the object
3. Constructor: this refers to the new instance
4. call(), apply(), bind(): this is explicitly set
5. Arrow function: this is lexically inherited

```
// 1. Global context
console.log(this); // window (in browser)

// 2. Object method
const user = { name: 'Aman', say: function() { console.log(this.name); } };
user.say(); // Aman

// 3. Constructor
function Person(name) { this.name = name; }
const p = new Person('Aman');
console.log(p.name); // Aman

// 4. call(), apply(), bind()
function greet() { console.log(this.name); }
const obj = { name: 'Aman' };
greet.call(obj); // Aman

// 5. Arrow function
const obj3 = { name: 'Aman', say: () => console.log(this.name) };
obj3.say(); // undefined
```

Top 50 JavaScript Interview Questions

@Aman_views

21. What is the difference between call(), apply(), and bind()?

- All three methods are used to call a function with a specific this value.
- The difference is in how arguments are passed and what they return.

Method	Arguments	Returns	Immediately Invokes?
call()	Passed individually	Result of the function	Yes
apply()	Passed as an array	Result of the function	Yes
bind()	Passed individually	New function	No

```
const person = { name: 'Aman' };

function greet(greeting, punctuation) {
  return greeting + ' ' + this.name + punctuation;
}

console.log(greet.call(person, 'Hello', '!'));  // Hello Aman!

console.log(greet.apply(person, ['Hi', '!']));  // Hi Aman!

const greetAman = greet.bind(person, 'Hey', '!');
console.log(greetAman());                       // Hey Aman!
```

22. How does this behave inside an arrow function?

- Arrow functions do not have their own this.
- They inherit this from the lexical (surrounding) scope where they are defined.
- Changing how the arrow function is called does not change this.

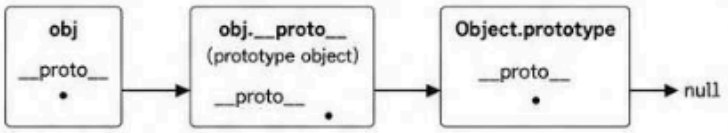
```
const obj = {
  name: 'Aman',
  regularFn: function() {
    console.log(this.name);           // Aman
  },
  arrowFn: () => {
    console.log(this.name);           // undefined (in browser)
  }
};

obj.regularFn();                      // Aman
obj.arrowFn();                        // undefined

const fn = obj.arrowFn;
fn();                                 // undefined (this doesn't change)
```

23. What is prototypal inheritance in JavaScript?

- JavaScript is prototype-based, not class-based.
- Objects inherit properties and methods from other objects through their prototype.
- Every object has an internal link to its prototype object.




```
obj.__proto__ === Object.prototype    // true
Object.prototype.__proto__ === null   // true
```

24. What is the prototype chain?

- When you access a property on an object, JavaScript first looks in the object itself.
- If not found, it looks in the object's prototype.
- This continues until the property is found or the end of the chain (null).

```
const arr = [1, 2, 3];
console.log(arr.length);                  // 3 (found in arr)
console.log(arr.toString());              // inherited from Array.prototype
console.log(arr.hasOwnProperty('length'); // true
console.log(arr.hasOwnProperty('toString'));// false

// Prototype chain of arr:
// arr -> Array.prototype -> Object.prototype -> null
```

25. What is the difference between __proto__ and prototype?

Aspect	__proto__	prototype
Type	Accessor property	Property
Used on	Instances (objects)	Constructor functions
Purpose	Points to the prototype of an object	The prototype object that will be inherited by instances
Where	Inside every object	Only on functions (constructors)
Example	obj.__proto__ === Person.prototype	function Person() {} Person.prototype.sayHello = function() {};
Standard	Non-standard (but widely supported)	Standard

```
function Person(name) {
  this.name = name;
}
Person.prototype.sayHello = function() {
  console.log('Hello ' + this.name);
};

const p1 = new Person('Aman');
console.log(p1.__proto__ === Person.prototype); // true
console.log(Person.prototype.isPrototypeOf(p1)); // true
```

26. What does the new keyword do internally?

1. Creates a new empty object.
2. Sets the **__proto__** of the new object to the constructor's prototype.
3. Binds **this** to the new object.
4. Executes the constructor function.
5. Returns the new object (unless constructor returns another object).

```
function Person(name) {
  this.name = name;
}

const p = new Person('Aman');
console.log(p.name);                  // Aman
console.log(p instanceof Person);     // true
```

Top 50 JavaScript Interview Questions

@Aman_views

27. What is the difference between Object.create() and new?

Aspect	Object.create()	new
Purpose	Creates a new object with the specified prototype object.	Creates an instance of a constructor function.
Constructor function	Not required.	Required.
Initializes this	No.	Yes (inside constructor).
Arguments	Takes prototype object.	Can pass arguments to constructor.
Use case	When you just want to set the prototype.	When you want to create instances with initialization.
Example	`const obj = Object.create(proto);`	`const obj = new Person('Aman');`

```
// Using Object.create()
const proto = {
  sayHi() { console.log('Hi'); }
};
const obj1 = Object.create(proto);
obj1.sayHi();     // Hi
console.log(obj1.hasOwnProperty('sayHi'));  // false

// Using new
function Person(name) {
  this.name = name;
}
Person.prototype.sayHi = function() {
  console.log('Hi');
};
const obj2 = new Person('Aman');
obj2.sayHi();
console.log(obj2.hasOwnProperty('sayHi'));  // false
console.log(obj2.name);                     // Aman
```

28. What is the difference between Object.freeze(), Object.seal(), and Object.preventExtensions()?

Method	What it does	Can add new properties?	Can delete existing properties?	Can modify existing properties?	Can change prototype?
Object.freeze()	Makes object immutable. No changes allowed.	No	No	No	No
Object.seal()	Seals the object. Existing properties can be modified but not added or deleted.	No	No	Yes	No
Object.preventExtensions()	Prevents adding new properties. Existing properties can be modified or deleted.	No	Yes	Yes	No

```
const obj1 = { a: 1 };
Object.freeze(obj1);
obj1.a = 10;         // fails silently (strict mode: error)
obj1.b = 2;          // fails
delete obj1.a;       // fails
console.log(obj1);   // { a: 1 }
```

```
const obj1 = { a: 1 };
Object.seal(obj2);
obj2.a = 10;         // allowed
obj2.b = 2;          // fails
delete obj2.a;       // fails
console.log(obj2);   // { a: 10 }
```

```
const obj3 = { a: 1 };
Object.preventExtensions(obj3);
obj3.a = 10;         // allowed
obj3.b = 2;          // fails
delete obj3.a;       // allowed
console.log(obj3);   // {} or { a: 10 } after deletion
```

29. What is the difference between for...in and for...of?

Feature	for...in	for...of
Purpose	Iterates over enumerable property keys of an object.	Iterates over iterable values (arrays, strings, maps, sets, etc.).
Returns	Keys (property names).	Values.
Works on	Objects.	Iterables (Array, String, Map, Set, NodeList, etc.).
Order	Not guaranteed.	Respects insertion order.

Example	`const obj = { a: 1, b: 2 };`
	`for (let key in obj) {`
	`  console.log(key);  // a, b`
	`}`
	
	`const arr = [10, 20, 30];`
	`for (let index in arr) {`
	`  console.log(index);  // 0, 1, 2 (indexes)`
	`}`
		`const arr = [10, 20, 30];`
		`for (let value of arr) {`
		`  console.log(value);  // 10, 20, 30`
		`}`
		
		`const str = 'Aman';`
		`for (let ch of str) {`
		`  console.log(ch);  // A, m, a, n`
		`}`

30. How do you check whether an object has a specific property?

- Using in operator (checks in object + prototype chain)
- Using hasOwnProperty() (checks only in object)
- Using Object.hasOwn() (ES2022+) (checks only in object)

```
const obj = { a : 1 };
obj.b = undefined;
const proto = { c: 3 };
Object.setPrototypeOf(obj, proto);

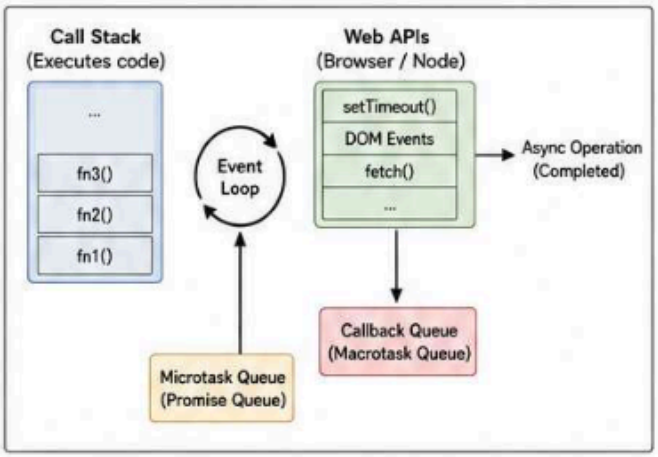
'a' in obj;                 // true
'c' in obj;                 // true (from prototype)
'b' in obj;                 // true (property exists with undefined)
"d" in obj;                 // false
obj.hasOwnProperty('a');    // true
obj.hasOwnProperty('c');    // false
Object.hasOwn(obj, 'b');    // true
Object.hasOwn(obj, 'c');    // false
```

Top 50 JavaScript Interview Questions

@Aman_views

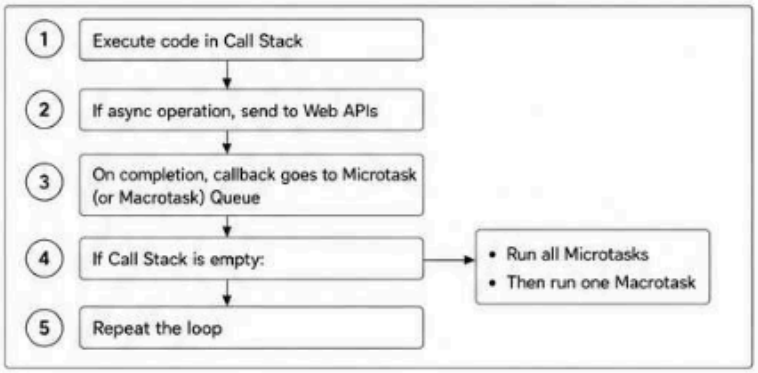
31. Is JavaScript really single-threaded? How does it handle asynchronous operations?

- Yes, JavaScript is single-threaded, which means it has only one call stack and can execute one task at a time.
- Asynchronous operations (like setTimeout, Promises, fetch, events) are handled using the Event Loop, Callback Queue, HTML APIs, Microtask Queue, and Call Stack.
- When an async operation is initiated, JS doesn't wait for it to finish. It continues executing the next task.
- Once the async operation is completed, its callback is placed in the appropriate queue.
- The Event Loop continuously checks the queues and pushes tasks to the Call Stack when it is empty.



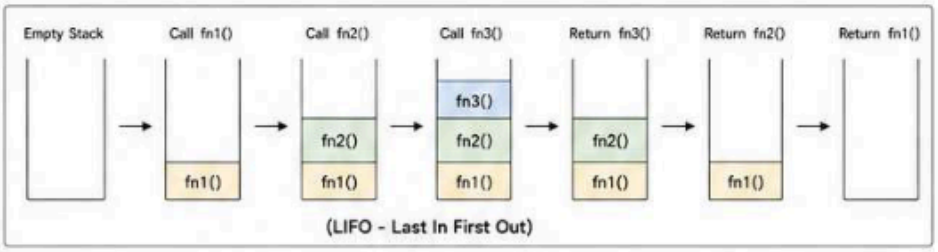
32. Explain the JavaScript Event Loop.

- The Event Loop is the mechanism that allows JavaScript to perform non-blocking I/O operations.
- It continuously checks:
  1. Call Stack is empty?
  2. If yes, it first processes all tasks in the Microtask Queue.
  3. Then, it takes the first task from the Macrotask (Callback) Queue and pushes it to the Call Stack.
  4. Repeat.



33. What is the Call Stack?

- The Call Stack is a data structure that keeps track of function calls.
- Whenever a function is called, it is pushed onto the stack.
- When the function returns, it is popped from the stack.
- JS uses LIFO (Last In First Out) principle.



34. What is the difference between the Microtask Queue and Macrotask Queue?

Feature	Microtask Queue	Macrotask Queue (Callback Queue)
Also called	Job Queue / Promise Queue	Task Queue
Examples	Promises (.then, .catch, .finally), queueMicrotask(), MutationObserver	setTimeout, setInterval, setImmediate, I/O, DOM events
Priority	Higher	Lower
Execution	All microtasks are executed before any macrotask.	One macrotask is executed after all microtasks are done.
When executed	When Call Stack is empty	After microtask queue is empty and Call Stack is empty
Starvation	If microtasks keep coming, macrotasks may be delayed.	Always wait for microtasks to finish.

```
// Example
console.log('Start');

setTimeout(() => console.log('setTimeout'), 0);

Promise.resolve().then(() => console.log('Promise 1'));
Promise.resolve().then(() => console.log('Promise 2'));

console.log('End');

// Output:
// Start
// End
// Promise 1
// Promise 2
// setTimeout
```

Explanation:

- 'Start' and 'End' are synchronous.
- Promises (microtasks) run before setTimeout (macrotask).

35. What is a Promise in JavaScript?

- A Promise is an object representing the eventual completion (or failure) of an asynchronous operation and its resulting value.
- It helps avoid callback hell and makes async code easier to write and maintain.
- A Promise has three states: pending, fulfilled, rejected.

```
const promise = new Promise((resolve, reject) => {
  const success = true;

  if (success) {
    resolve('Operation successful');
  } else {
    reject('Something went wrong');
  }
});

promise
  .then(result => console.log(result))    // Operation successful
  .catch(error => console.log(error));    // (if rejected)
```

Top 50 JavaScript Interview Questions

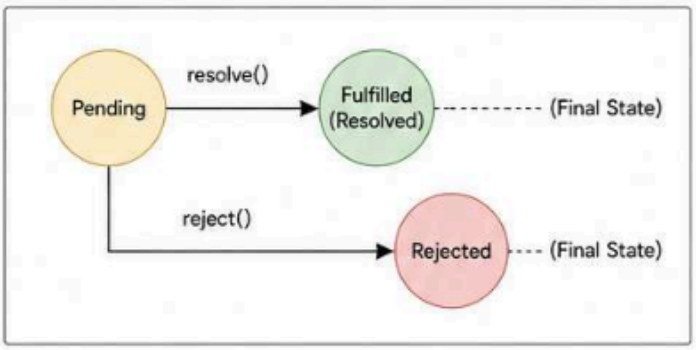
@Aman_views

36. What are the different states of a Promise?

- A Promise can be in one of these states:

State	Description	Transition
Pending	Initial state. Neither fulfilled nor rejected.	Can change to Fulfilled or Rejected.
Fulfilled (Resolved)	Operation completed successfully.	Final state.
Rejected	Operation failed.	Final state.

- Once a Promise is fulfilled or rejected, it is immutable and cannot change to another state.



37. What is the difference between Promise.all(), Promise.allSettled(), Promise.race(), and Promise.any()?

Method	Description	Resolves When	Rejects When	Return Value
Promise.all()	Waits for all promises to fulfill.	All promises are fulfilled.	If any promise rejects.	Array of all fulfillment values (in order).
Promise.allSettled()	Waits for all promises to either fulfill or reject.	All promises are settled (fulfilled or rejected).	Never rejects.	Array of objects with status and value/reason.
Promise.race()	Settles as soon as one promise settles.	The first promise is fulfilled.	The first promise is rejected.	Value or reason of the first settled promise.
Promise.any()	Resolves as soon as one promise fulfills.	The first promise is fulfilled.	If all promises reject.	Value of the first fulfilled promise.

1. Promise.all()

```
const p1 = Promise.resolve(1);
const p2 = Promise.resolve(2);
const p3 = Promise.resolve(3);

Promise.all([p1, p2, p3])
 .then(res => console.log(res))
.catch(err => console.log(err));
// Output: [1, 2, 3]
```

2. Promise.allSettled()

```
const p1 = Promise.resolve(1);
const p2 = Promise.reject('Err');

Promise.allSettled([p1, p2])
 .then(res => console.log(res));
/* Output:
[
  { status: 'fuffilled', value: 1 },
  { status: 'rejected', reason: 'Err'}
]
*/
```

3. Promise.race()

```
const p1 = new Promise(res =>
 setTimeout(() => res('First'), 1000));
const p2 = new Promise(res =>
 setTimeout(() => res('Second'), 500));

Promise.race([p1, p2])
.then(res => console.log(res));
// Output: Second
```

4. Promise.any()

```
const p1 = Promise.reject('Err1');
const p2 = Promise.reject('Err2');
const p3 = Promise.resolve('Success'));

Promise.any([p1, p2, p3])
 .then(res => console.log(res))
 .catch(err => console.log(err));
// Output: Success
```

38. What is async/await and how does it work with Promises?

- async/await is syntactic sugar over Promises.
- async makes a function return a Promise.
- await pauses the execution of the async function until the Promise is resolved or rejected.
- It makes asynchronous code look and behave like synchronous code.

```
// Without async/await (using Promises)
function fetchData() {
  return fetch('https://api.example.com/data')
    .then(res => res.json())
    .then(data => console.log(data))
    .catch(err => console.log(err));
}

// With async/await
async function fetchData() {
  try {
    const res = await fetch('https://api.example.com/data');
    const data = await res.json();
    console.log(data);
  } catch (err) {
    console.log(err);
}
fetchData();
```

39. How do you handle errors in asynchronous JavaScript?

- Use try...catch with async/await.
- Use .catch() with Promises.
- Handle rejected Promises properly to avoid unhandled promise rejections.

Using async/await

```
async function getUser() {
  try {
    const res = await fetch('https://api.example.com/user');
    if (!res.ok) throw new Error('Failed to fetch');
    const data = await res.json();
    console.log(data);
  } catch (err) {
    console.log('Error:', err.message);
  }
}
getUser();
```

Using .catch()

```
fetch('https://api.example.com/user')
 .then(res => {
   if (!res.ok) throw new Error('Failed to fetch');
   return res.json();
 })
 .then(data => console.log(data))
.catch(err => console.log('Error:', err.message));
```

40. Predict the output of a program containing setTimeout(), Promises, and synchronous code.

```
console.log('1 - Start');

setTimeout(() => console.log('2 - setTimeout'), 0);

Promise.resolve().then(() => console.log('3 - Promise 1'));
Promise.resolve().then(() => console.log('4 - Promise 2'));

console.log('5 - End');
```

Explanation:

1. Synchronous code runs first: '1 - Start' and '5 - End'
2. Then microtasks (Promises) run in order of queue: '3 - Promise 1', '4 - Promise 2'
3. Finally, macrotask (setTimeout) runs: '2 - setTimeout'

Output:

```
1 - Start
5 - End
3 - Promise 1
4 - Promise 2
2 - setTimeout
```

Top 50 JavaScript Interview Questions

@Aman_views

41. What are destructuring, spread, and rest operators in JavaScript?

- **Destructuring:** Extract values from arrays or properties from objects into variables.
- **Spread (...):** Expands an iterable (array/object) into individual elements.
- **Rest (...):** Collects multiple elements into a single array or object.

```js
// 1. Destructuring
const arr = [10, 20, 30];
const [a, b, c] = arr;
console.log(a, b, c); // 10 20 30

const obj = { name: 'Aman', age: 25 };
const { name, age } = obj;
console.log(name, age); // Aman 25
```

```js
// 2. Spread
// In array
const arr1 = [1, 2];
const arr2 = [...arr1, 3, 4];
console.log(arr2); // [1, 2, 3, 4]

// In object
const obj1 = { a: 1, b: 2 };
const obj2 = { ...obj1, c: 3 };
console.log(obj2);  // { a: 1, b: 2, c: 3 }
```

```js
// 3. Rest
// In array
const [first, ...rest] = [1, 2, 3, 4, 5];
console.log(first); // 1
console.log(rest); // [2, 3, 4, 5]

// In object
const { name: n, ...others } = { name: 'Aman',
age: 25, city: 'Delhi' };
console.log(others); // { age: 25, city: 'Delhi' }
```

42. What is the difference between map(), filter(), and reduce()?

Method	Purpose	Returns	Modifies Original Array	Example
map()	Transforms each element using a function.	New array with transformed elements.	No	`arr.map(x => x * 2)`
filter()	Filters elements based on a condition.	New array with elements that pass the condition.	No	`arr.filter(x => x > 2)`
reduce()	Reduces array to a single value by applying a function accumulatively.	Single value.	No	`arr.reduce((acc, x) => acc + x, 0)`

```js
const arr = [1, 2, 3, 4];
// map(): multiply each by 2
const mapped = arr.map(x => x * 2);
console.log(mapped); // [2, 4, 6, 8]
```

```js
// filter(): keep elements > 2
const filtered = arr.filterr(x => x > 2);
console.log(filtered); // [3, 4]
```

```js
// reduce(): sum of all elements
const sum = arr.reduce((acc, curr) => acc + curr, 0);
console.log(sum); // 10
```

43. What is the difference between map() and forEach()?

Feature	map()	forEach()
Purpose	Transforms each element and creates a new array.	Executes a function for each element, no new array.
Returns	New array.	undefined.
Can be chained	Yes.	No.
Use case	When you want to transform data.	When you want to perform side effects (like logging, updating).

```js
const arr = [1, 2, 3];

// map()
const doubled = arr.map(x => x * 2);
console.log(doubled); // [2, 4, 6]

// forEach()
arr.forEach(x => console.log(x * 2));
// Output:
// 2
// 4
// 6

const res = arr.forEach(x => x * 2);
console.log(res); // undefined
```

44. What is debouncing and where is it used?

- Debouncing ensures a function is executed only after a certain delay has passed since the last time it was invoked.
- If the function is called again before the delay, the timer resets.
- Commonly used in: search input, window resize, button clicks, auto-save, API calls.

```js
// Debounce function
function debounce(fn, delay) {
  let timer;
  return function(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}
// Usage
window.addEventListener('resize', debounce(() => {
  console.log('Window resized');
}, 100));
```

How it works:

If the event keeps firing (e.g., resize), the function runs only once after 300ms of inactivity.

45. What is throttling and where is it used?

- Throttling ensures a function is executed at most once in a given time interval.
- It ignores subsequent calls that happen within the interval.
- Commonly used in: scroll events, mouse move, window resize, rate-limited API calls.

```js
// Throttle function
function throttle(fn, limit) {
  let lastTime = 0;
  return function(...args) {
    const now = Date.now();
    if (now - lastTime >= limit) {
      lastTime = now;
      fn.apply(this, args);
    }
  };
}
// Usage
window.addEventListener('scroll', throttle(() => {
  console.log('Scroll event');
}, 1000));
```

How it works:

The function executes immediately, then ignores calls for the next 1000ms. After that, it can run again.

Top 50 JavaScript Interview Questions

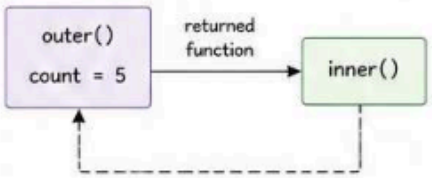
46. What are JavaScript Closures?

- A closure is a function that remembers the variables from its outer (lexical) scope even after the outer function has finished execution.
- Closures are created every time a function is created.
- Useful for data hiding, currying, private variables, etc.

```
function outer(x) {
  let count = x;
  return function inner() {
    count++;
    console.log(count);
  }
}

const fn = outer(5);
fn();  // 6
fn();  // 7
fn();  // 8
```

How it works:



inner() remembers count from outer() even after outer() has finished.

47. What is the difference between == and ===?

Operator	Type	Comparison	Example	Result
==	Abstract Equality	Compares values after type coercion	'5' == 5	true
===	Strict Equality	Compares values without type coercion	'5' === 5	false

Key Point:
Always prefer === unless you have a specific reason to use ==.

48. What is Hoisting in JavaScript?

- Hoisting is JavaScript's default behavior of moving declarations to the top of their scope before code execution.
- Only declarations are hoisted, not initializations.
- var is hoisted and initialized with undefined. let and const are hoisted but not initialized (Temporal Dead Zone).

var (hoisted + initialized)

```
console.log(a);  // undefined
var a = 10;
console.log(a);  // 10
```

let / const (hoisted, not initialized)

```
console.log(b);  // ReferenceError
let b = 20;

console.log(c);  // ReferenceError
const c = 30;
```

☆ Always declare variables before using them.

49. What is the difference between null and undefined?

Feature	null	undefined
Type	object	undefined
Meaning	Intentional absence of any value	Variable declared but not assigned
Assigned by	Developer	JavaScript engine
Example	let a = null;	let b; // or let b; (not assigned)
Check	a === null	b === undefined

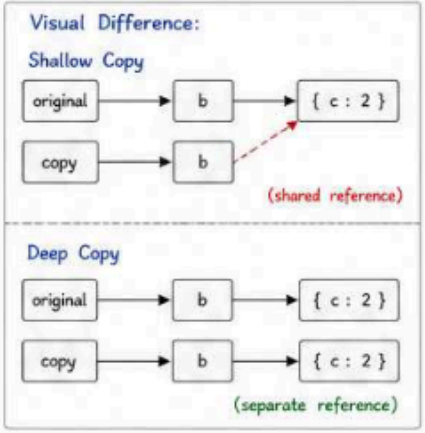
```
let a = null;
let b;

console.log(a);            // null
console.log(b);            // undefined
console.log(typeof a);     // object
console.log(typeof b);     // undefined
```

50. What is the difference between Shallow Copy and Deep Copy?

Feature	Shallow Copy	Deep Copy
Definition	Copies only top-level properties. Nested objects are shared.	Copies all levels of properties. No references are shared.
Affects Original	Changes in nested object affect both copies.	Changes in nested object do not affect the copy.
Performance	Faster	Slower
Methods	Object.assign(), Spread operator (...)	JSON.parse(JSON.stringify()), structuredClone(), Libraries (lodash.cloneDeep)
Example	const original = { a: 1, b: { c: 2 } }; const shallow = { ...original }; shallow.b.c = 99; console.log(original.b.c); // 99 (changed)	const original = { a: 1, b: { c: 2 } }; const deep = JSON.parse(JSON.stringify(original)); deep.b.c = 99; console.log(original.b.c); // 2 (unchanged)

Visual Difference:



Pro Tip: *Practice by coding these concepts. Understanding by doing is the best way to remember!*